

**Nirma University
Institute of Technology**



Course Code :- 6CS105CC26
Course Name :- Computer Network

Practical - 1

Student Name
Thakor Vinay

Student Roll No.
26MCA061

Faculty In Charge
Dr. Lata Gohil

Bit Stuffing – Sender and Receiver

Aim

To write a sender and receiver side program to demonstrate **bit stuffing**, where a frame contains a header, payload, trailer, and starting/ending flags. The payload is read from and written back to a file.

Concept

Bit stuffing is a technique used in data communication to prevent the data from being confused with the special **flag sequence** used to identify the beginning and end of a frame.

The starting and ending flag used in this program is:

01111110

Whenever **five consecutive 1s** are found in the payload, the sender automatically inserts a **0** after them.

At the receiver side, this extra 0 is removed to obtain the original payload.

Frame Format

The frame generated by the sender is:

Flag | Header | Stuffed Payload | Trailer | Flag

In this program:

- **Flag:** 01111110
- **Header:** 10101010
- **Trailer:** 01010101
- **Payload:** Binary data entered by the user

These values are defined in both the sender and receiver programs.

Sender Side

The sender program performs the following steps:

1. Takes binary data as input from the user.
2. Applies bit stuffing to the payload.
3. After every five consecutive 1s, it inserts a 0.
4. Creates the complete frame using the flag, header, stuffed payload, and trailer.
5. Writes the generated frame into `data.txt`.

The main bit stuffing function is `bitStuff()`.

For example:

Original payload:

1111111

After bit stuffing:

11111011

The extra 0 is inserted after five consecutive 1s.

The sender creates the frame using:

FLAG + HEADER + stuffedPayload + TRAILER + FLAG

The generated frame is then stored in `data.txt`.

```
// Data will be stored in data.txt
// Bits will be stuffed in the payload and sent to the receiver. The receiver
// will unstuff the bits and write the original data back to the file.
// | Flag | Header | Payload | Trailer | Flag |
import java.io.*;
import java.util.*;
import java.nio.file.*;

public class Sender {
    static final String FLAG = "01111110";
    static final String HEADER = "10101010";
    static final String TRAILER = "01010101";

    public static String bitStuff(String payload) {
        StringBuilder stuffed = new StringBuilder();
        int count = 0;

        for (char bit : payload.toCharArray()) {
            stuffed.append(bit);
            if (bit == '1') {
                count++;
                if (count == 5) {
                    stuffed.append('0');
                    count = 0;
                }
            }
        }
        return stuffed.toString();
    }
}
```

```

    }

    }
    return stuffed.toString();
}

public static void main(String args[]) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter data in binary: ");
    String data = sc.nextLine();
    String stuffedPayload = bitStuff(data);
    System.out.println("Stuffed payload: " + stuffedPayload);

    String frame = FLAG + HEADER + stuffedPayload + TRAILER + FLAG;
    System.out.println("Frame to be sent: " + frame);

    try {
        Files.write(Paths.get("data.txt"), frame.getBytes());
        System.out.println("Frame written to data.txt");
    } catch (IOException e) {
        System.out.println("An error occurred while writing to the file.");
        e.printStackTrace();
    }

    sc.close();
}
}

```

Receiver Side

The receiver reads the frame from `data.txt`.

It performs the following steps:

1. Reads the frame from the file.
2. Checks whether the frame starts and ends with the correct flag `01111110`.
3. Removes the flag, header, and trailer to extract the stuffed payload.
4. Applies bit unstuffing.
5. Removes the `0` inserted after every five consecutive `1`s.
6. Writes the original payload back to `data.txt`.

The receiver uses the `bitUnstuff()` function for removing the stuffed bits.

Example

Suppose the input payload is:

```
11111011111
```

At Sender

The sender finds five consecutive `1`s and inserts `0` after each group.

Stuffed payload:

```
1111100111110
```

The frame becomes:

```
01111110 | 10101010 | 1111100111110 | 01010101 | 01111110
```

This complete frame is written to `data.txt`.

At Receiver

The receiver extracts the stuffed payload:

```
1111100111110
```

It removes the inserted `0`s and obtains:

```
11111011111
```

The original payload is then written back to `data.txt`.

```

import java.io.*;
import java.nio.file.*;

public class Receiver {
    static final String FLAG = "01111110";
    static final String HEADER = "10101010";
    static final String TRAILER = "01010101";

    public static String bitUnstuff(String stuffedPayload) {
        StringBuilder unstuffed = new StringBuilder();
        int count = 0;

        for (int i = 0; i < stuffedPayload.length(); i++) {
            char bit = stuffedPayload.charAt(i);
            unstuffed.append(bit);
            if (bit == '1') {
                count++;
                if (count == 5) {
                    if (i + 1 < stuffedPayload.length() &&
stuffedPayload.charAt(i + 1) == '0') {
                        i++;
                    }
                    count = 0;
                }
            } else {
                count = 0;
            }
        }
        return unstuffed.toString();
    }

    public static void main(String args[]) {
        try {
            String frame = new String(Files.readAllBytes(Paths.get("data.txt")));
            System.out.println("Received frame: " + frame);

            if (frame.startsWith(FLAG) && frame.endsWith(FLAG)) {
                String payload = frame.substring(FLAG.length() + HEADER.length(),
frame.length() - FLAG.length() - TRAILER.length());
                System.out.println("Extracted stuffed payload: " + payload);

                String unstuffedPayload = bitUnstuff(payload);
                System.out.println("Unstuffed payload: " + unstuffedPayload);

                Files.write(Paths.get("data.txt"), unstuffedPayload.getBytes());
                System.out.println("Original data written to data.txt");
            } else {
                System.out.println("Invalid frame received.");
            }
        } catch (IOException e) {
            System.out.println("An error occurred while reading from the file.");
            e.printStackTrace();
        }
    }
}

```

File Handling

The program uses the Java `Files` class for file operations.

- **Sender:** writes the generated frame to `data.txt`.
- **Receiver:** reads the frame from `data.txt` and writes the unstuffed/original payload back to the same file.

The initial `data.txt` contains binary data used by the program.

Example :

```
Vinay@DEVLOQ in repo: Nirma_SEM-1\CN\prac_2 on P main [x!?] via 4 v23 took 0s
> cd "e:\Codes\Nirma_SEM-1\CN\prac_2\" ; if ($?) { javac Sender.java } ; if ($?) { java Sender } ; if ($?) { del
• Sender.class }
Enter data in binary: 1111111111
Stuffed payload: 111101111101
Frame to be sent: 0111110101010111110111110101010101111110
Frame written to data.txt

Vinay@DEVLOQ in repo: Nirma_SEM-1\CN\prac_2 on P main [x!?] via 4 v23 took 2s
> cd "e:\Codes\Nirma_SEM-1\CN\prac_2\" ; if ($?) { javac Receiver.java } ; if ($?) { java Receiver } ; if ($?) {
• del Receiver.class }
Received frame: 0111110101010111110111110101010101111110
Extracted stuffed payload: 111101111101
Unstuffed payload: 1111111111
Original data written to data.txt

Vinay@DEVLOQ in repo: Nirma_SEM-1\CN\prac_2 on P main [x!?] via 4 v23 took 0s
>
```

Conclusion

Thus, the sender and receiver programs successfully demonstrate **bit stuffing and bit unstuffing**. The sender inserts a 0 after every five consecutive 1s, creates a frame with header, payload, trailer, and flags, and stores it in a file. The receiver extracts the payload, removes the stuffed bits, and writes the original data back to the file.